Sommario

ITERAZIONE 1 2
1. Introduzione 2
2. Analisi 2
2.1. Modello di Dominio 2
2.2. Caso d'uso UC1: REGISTRAZIONE E VERIFICA ETÀ 4
2.2.1. Diagramma di Sequenza di Sistema (SSD) 4
2.2.2. Contratti delle operazioni 4
2.3. Caso d'uso UC2: INSERIMENTO EVENTO 5
2.3.1. Diagramma di Sequenza di Sistema 5
2.3.2. Contratti delle operazioni 6
2.4. Caso d'uso UC3: RICERCA EVENTO 7
2.4.1. Diagramma di Sequenza di Sistema 7
2.4.2. Contratti delle operazioni 8
3. Progettazione 8
3.1. Caso d'uso di avviamento, diagramma di comunicazione 8
3.2. Diagrammi di sequenza UC1 9
3.2.1. inserisciDati() 10
3.2.2. confermaRegistrazione() 10
3.3. Diagrammi di Sequenza UC2 10
3.3.1. inserisciEvento() 11
3.3.2. selezionaTipologia() 11
3.3.3. inserisciServizioDettagli() 11
3.3.4. confermaEvento() 12
3.4. Diagrammi di Sequenza UC3 12
3.4.1. cercaEvento() 12
3.4. Diagramma delle Classi (DCD) 13
4. Testing 13
4.1. Individuazione dei casi di test 13

ITERAZIONE 1

1. Introduzione

La prima iterazione si basa sull'analisi e sulla progettazione dei primi tre casi d'uso.

L'obiettivo di questa iterazione, così come le successive, è quello di andare a progettare, in maniera iterativa, il Sistema pensato, presentando quali requisiti si considerano per la presente iterazione, raffinando i vari elaborati e discutendo l'applicazione di eventuali pattern applicati. Verranno, inoltre, presentati i vari test effettuati per garantire il corretto funzionamento del codice dell'applicazione.

In particolare, per questa prima iterazione, si andranno a considerare i seguenti casi d'uso:

- UC1: REGISTRAZIONE E VERIFICA ETÀ
- UC2: INSERIMENTO EVENTO
- UC3: RICERCA EVENTO

2. Analisi

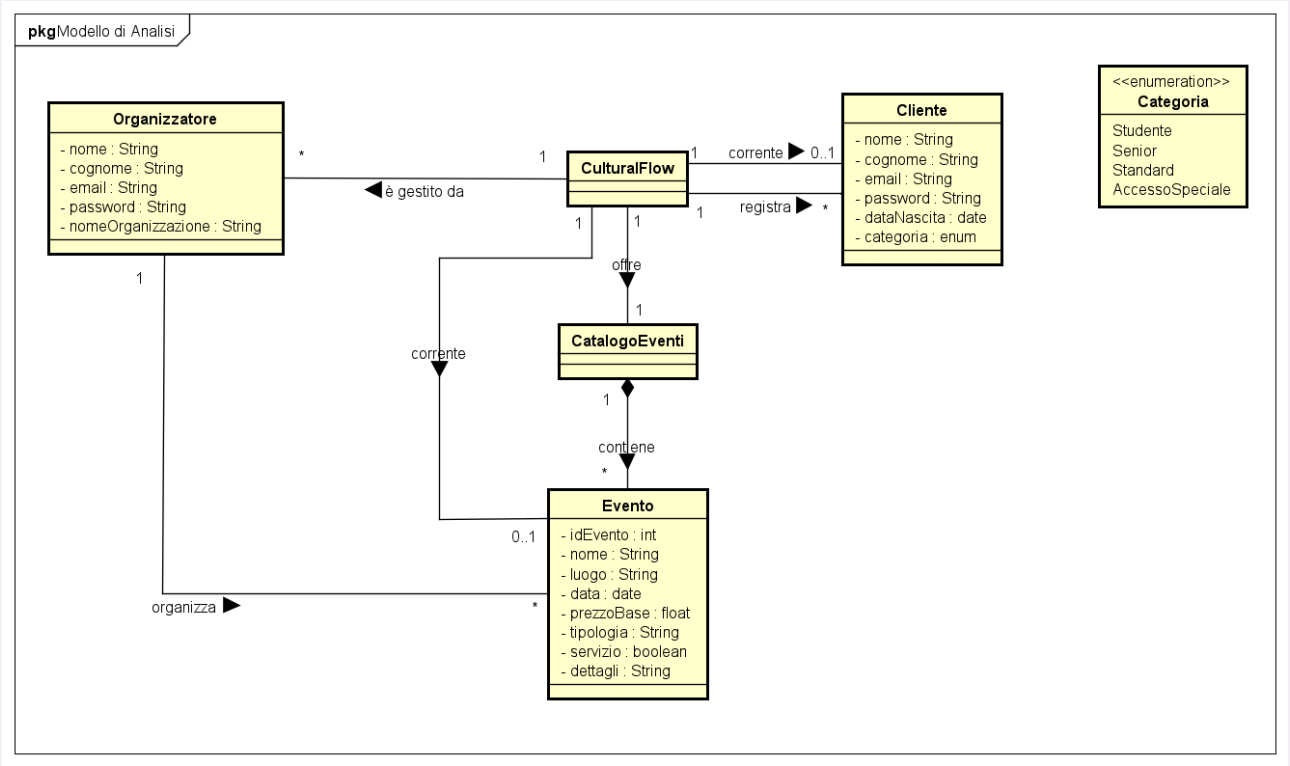
In questa prima fase vedremo nel dettaglio il Modello di Dominio e successivamente, per ogni caso d'uso considerato, i diagrammi di sequenza di sistema, che permettono di descrivere visivamente il flusso delle operazioni scambiate tra attori e sistema, e i contratti delle operazioni, che specificano formalmente le operazioni individuate precedentemente.

2.1. Modello di Dominio

Le classi concettuali identificate durante questa prima iterazione sono le seguenti:

- Organizzatore: attore che interagisce con il sistema;
- CulturalFlow: rappresenta il sistema;
- CatalogoEventi: classe concettuale contenitore, contiene gli eventi inseriti e gestiti dal sistema;
- Evento: rappresenta uno degli eventi inseriti e gestiti dal sistema con le relative informazioni;
- Cliente: fruitore del sistema CulturalFlow, che può effettuare la registrazione al sistema e acquistare biglietti per vari eventi.

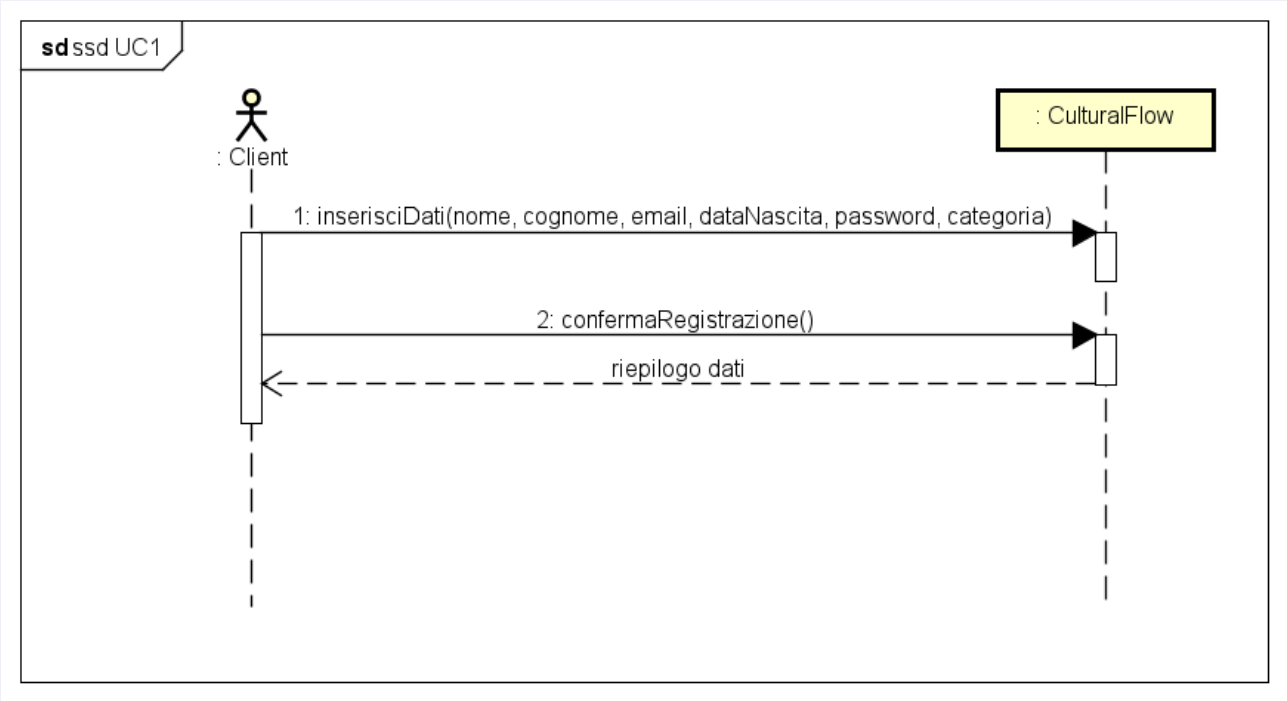
Tenendo conto di associazioni e attributi vari, analizzando i primi tre casi d'uso, è stato ricavato il seguente Modello di Dominio:


pkgModello di Analisi
Organizzatore
- nome : String
- cognome : String
- email : String
- password : String
- nomeOrganizzazione : String
CulturalFlow
Cliente
- nome : String
- cognome : String
- email : String
- password : String
- dataNascita : date
- categoria : enum
<<enumeration>>
Categoria
Studente
Senior
Standard
AccessoSpeciale
CatalogoEventi
Evento
- idEvento : int
- nome : String
- luogo : String
- data : date
- prezzoBase : float
- tipologia : String
- servizio : boolean
- dettagli : String
è gestito da
corrente
registra
offre
contiene
organizza
corrente
*
1
1
1
0..1
1
1
1
1
*
1
1
*
0..1
*

2.2. Caso d'uso UC1: REGISTRAZIONE E VERIFICA ETÀ

2.2.1. Diagramma di Sequenza di Sistema (SSD)

Il seguente diagramma di sequenza di sistema mostra l'interazione tra il cliente e il sistema con l'obiettivo di effettuare una registrazione.



2.2.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

CONTRATTO 1, UC1

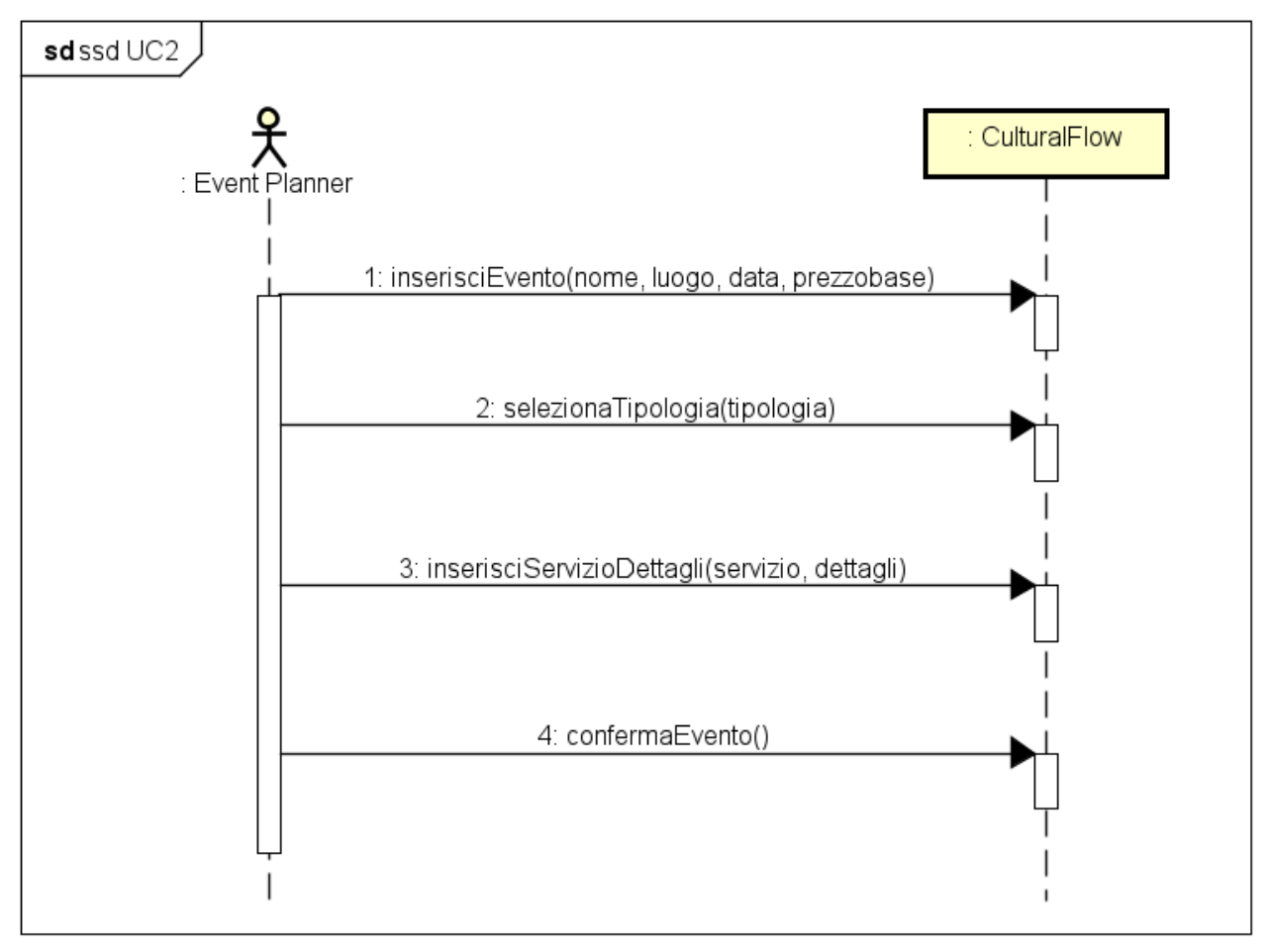
Operazione:	inserisciDati(nome:String, cognome:String, email:String, dataNascita:date, password:String, categoria:enum)
Riferimenti:	Caso d'uso: Registrazione e verifica età
Pre-condizioni:	-
Post-condizioni:	- è stata creata una nuova istanza c di Cliente; - sono stati inizializzati gli attributi di c; - il Cliente c è stato associato a CulturalFlow tramite l'associazione "corrente".

CONTRATTO 2, UC1	
Operazione:	confermaRegistrazione()
Riferimenti:	Caso d'uso: Registrazione e verifica età
Pre-condizioni:	- è in corso la registrazione di un nuovo Cliente
Post-condizioni:	- il Cliente è stato associato a CulturalFlow tramite l'associazione "registra".

2.3. Caso d'uso UC2: INSERIMENTO EVENTO

2.3.1. Diagramma di Sequenza di Sistema

Il seguente SSD mostra l'interazione tra l'Organizzatore e il Sistema al fine di inserire un nuovo evento.



2.3.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

CONTRATTO 1, UC2:

Operazione:	inserisciEvento(nome:String, luogo:String, data:date, prezzobase:float)
Riferimenti:	Caso d'uso: Inserimento Evento
Pre-condizioni:	-
Post-condizioni:	- è stata creata una nuova istanza e di Evento; - sono stati inizializzati gli attributi di e; - e è stato associato a CulturalFlow tramite l'associazione "corrente".

CONTRATTO 2, UC2:

Operazione:	selezionaTipologia(tipologia:String)
Riferimenti:	Caso d'uso: Inserimento Evento
Pre-condizioni:	- è in corso l'inserimento dell'Evento e
Post-condizioni:	- è stato inizializzato l'attributo "tipologia" di e.

CONTRATTO 3, UC2:

Operazione:	inserisciServizioDettagli(servizio:boolean, dettagli:String)
Riferimenti:	Caso d'uso: Inserimento Evento
Pre-condizioni:	- è in corso l'inserimento dell'Evento e
Post-condizioni:	- sono stati inizializzati gli attributi "servizio" e "dettagli" di e.

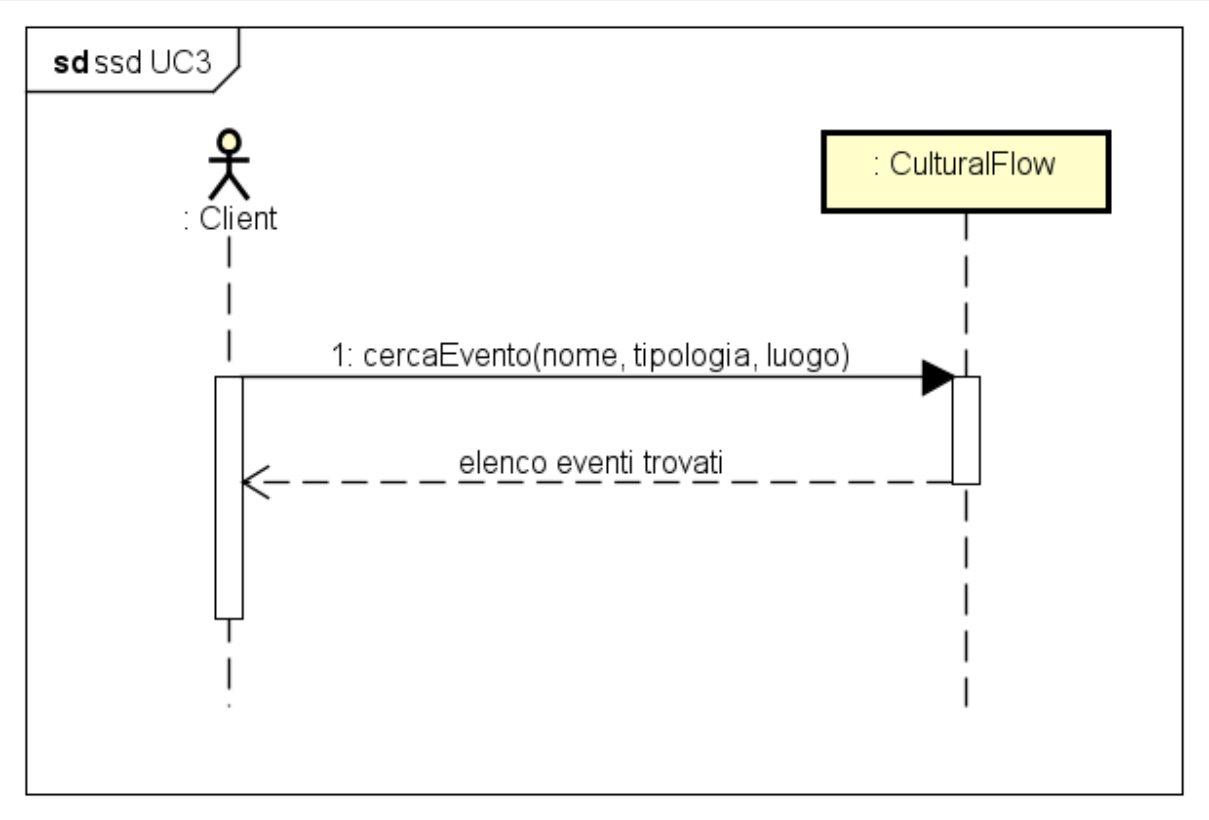
CONTRATTO 4, UC2:

Operazione:	confermaInserimento()
Riferimenti:	Caso d'uso: Inserimento Evento
Pre-condizioni:	- è in corso l'inserimento dell'Evento e
Post-condizioni:	- è stata associata l'istanza e di Evento corrente a CatalogoEventi tramite l'associazione "contiene"; - è stata associata l'istanza e di Evento corrente a Organizzatore tramite l'associazione "organizza".

2.4. Caso d'uso UC3: RICERCA EVENTO

2.4.1. Diagramma di Sequenza di Sistema

Il seguente SSD mostra l'interazione tra il Cliente e il Sistema al fine di cercare alcuni eventi.



2.4.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

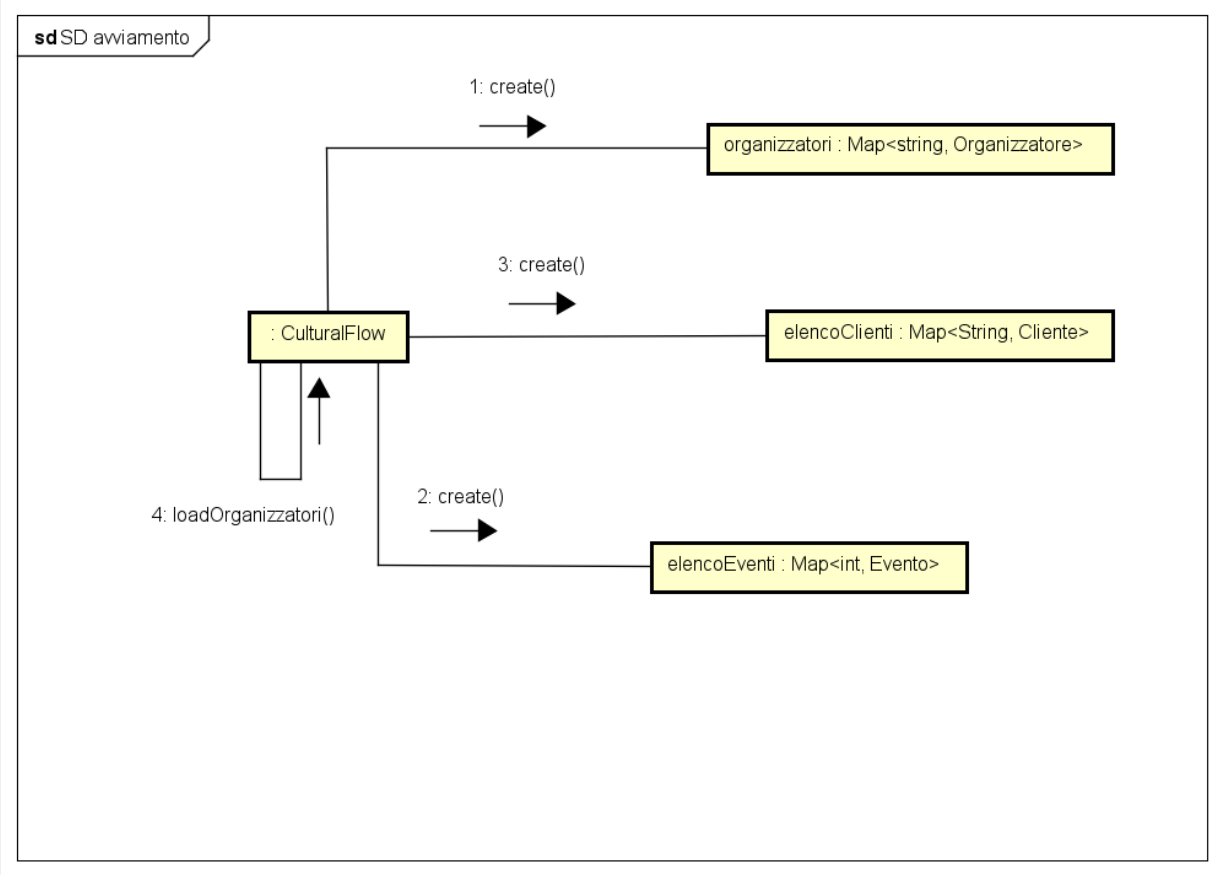
CONTRATTO 1, UC3:	
Operazione:	cercaEvento(nome:String tipologia:String, luogo:String)
Riferimenti:	Caso d'uso: Ricerca Evento
Pre-condizioni:	-
Post-condizioni:	- è stata generata una collezione di riferimenti agli oggetti Evento; - una o più istanze e di Evento vengono recuperate tramite i parametri passati; - e viene associata alla lista elencoEventiDisponibili; - viene ritornato l'elenco degli eventi disponibili.

3. Progettazione

In questa fase ci si concentra sui diagrammi di sequenza/comunicazione, che mostrano come gli oggetti collaborano tra loro, e sul Diagramma delle Classi di Progetto (DCD) che ci permette di ottenere una rappresentazione statica e dettagliata del sistema CulturalFlow.

3.1. Caso d'uso di avviamento, diagramma di comunicazione

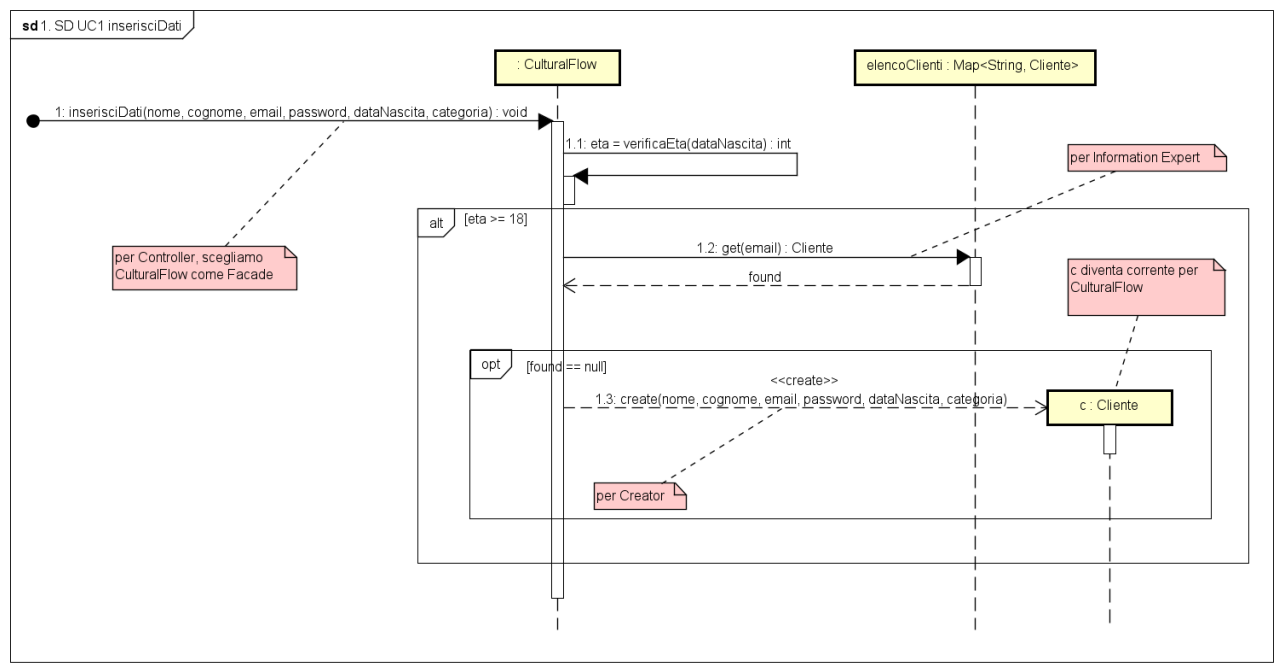
Per funzionare correttamente, il sistema dovrà seguire dei passaggi iniziali di configurazione.



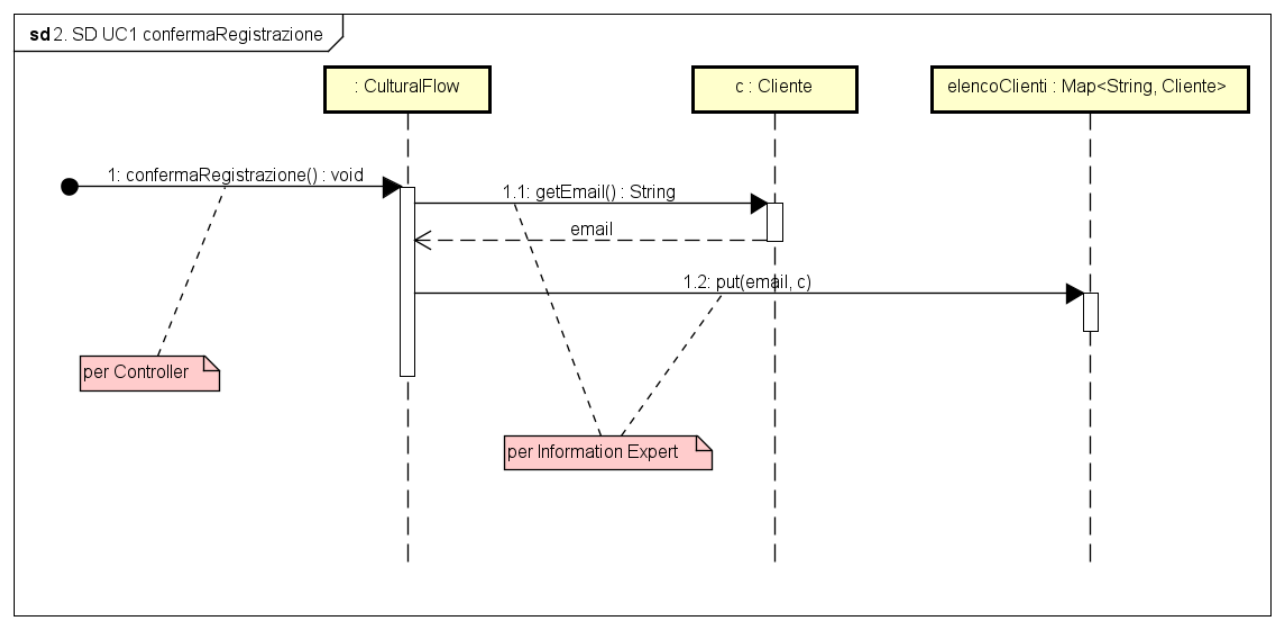
3.2. Diagrammi di sequenza UC1

In questi primi diagrammi di sequenza, relativi al caso d'uso UC1, vengono applicati diversi pattern GRASP. Il primo da individuare è il *pattern Controller*. Essendo che stiamo considerando un sistema poco complesso con poche operazioni di sistema, si è scelto di utilizzare un *Facade Controller* che è rappresentato proprio dal sistema CulturalFlow. Il sistema utilizza CulturalFlow come unico punto di accesso per le operazioni provenienti dall'interfaccia utente. Il sistema tiene, inoltre, traccia di tutti i Clienti che sono iscritti al suo interno, attraverso una mappa dei Clienti. Applicando il *pattern Creator*, il Sistema ha, dunque, il compito di creare nuovi oggetti di tipo Cliente. È stato utilizzato anche il *pattern Information Expert* per trovare eventuali Clienti già iscritti, permettendoci così di assegnare le responsabilità alle classi che possiedono le informazioni necessarie per portarle a termine.

3.2.1. inserisciDati()



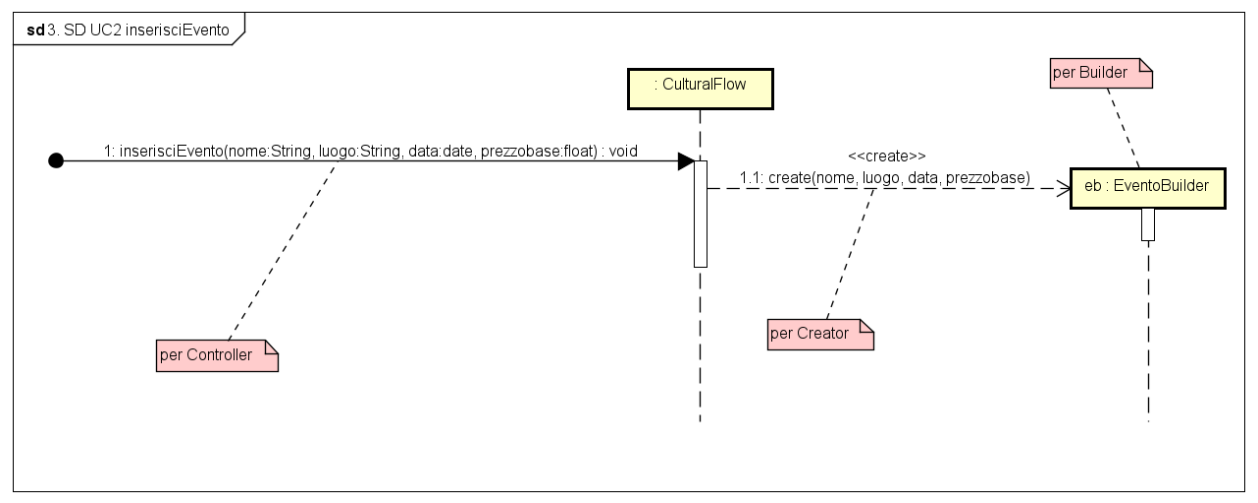
3.2.2. confermaRegistrazione()



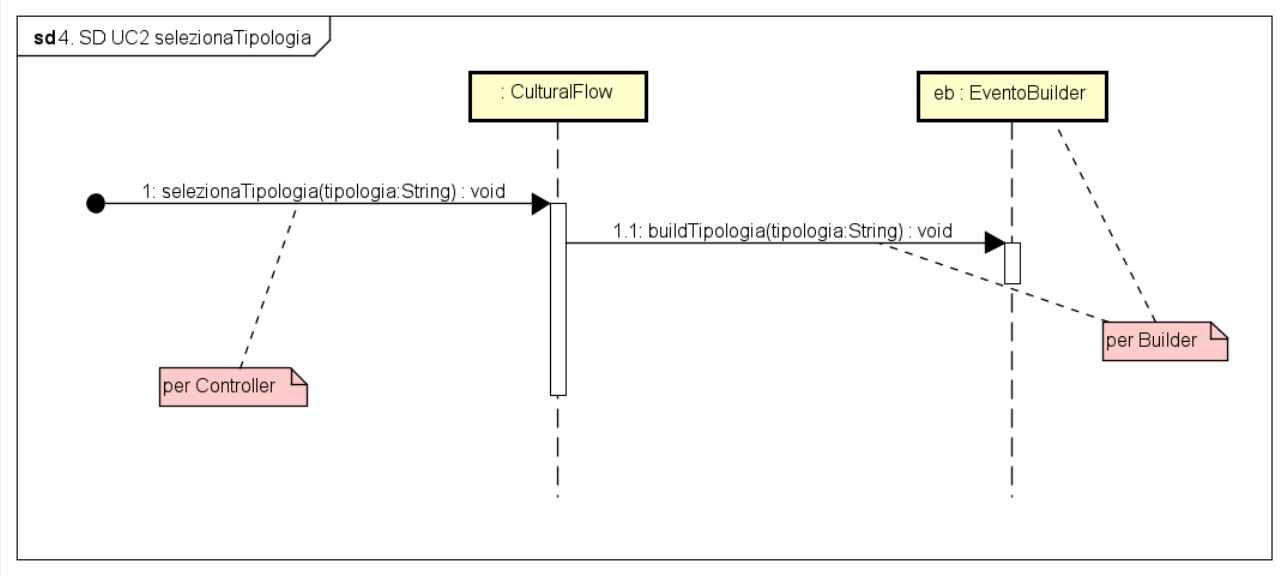
3.3. Diagrammi di Sequenza UC2

In questo secondo caso d'uso, viene applicato il *pattern Builder*, che è uno dei pattern creazionali GoF. Questo pattern viene applicato per la creazione dell'oggetto complesso Evento. Questo pattern ci permette di costruire l'oggetto passo dopo passo, separando la logica di costruzione dalla rappresentazione finale del prodotto (l'evento). Anche in questo caso è CulturalFlow che avrà il compito di creare le istanze di Evento, chiaramente tramite il supporto di un builder dedicato.

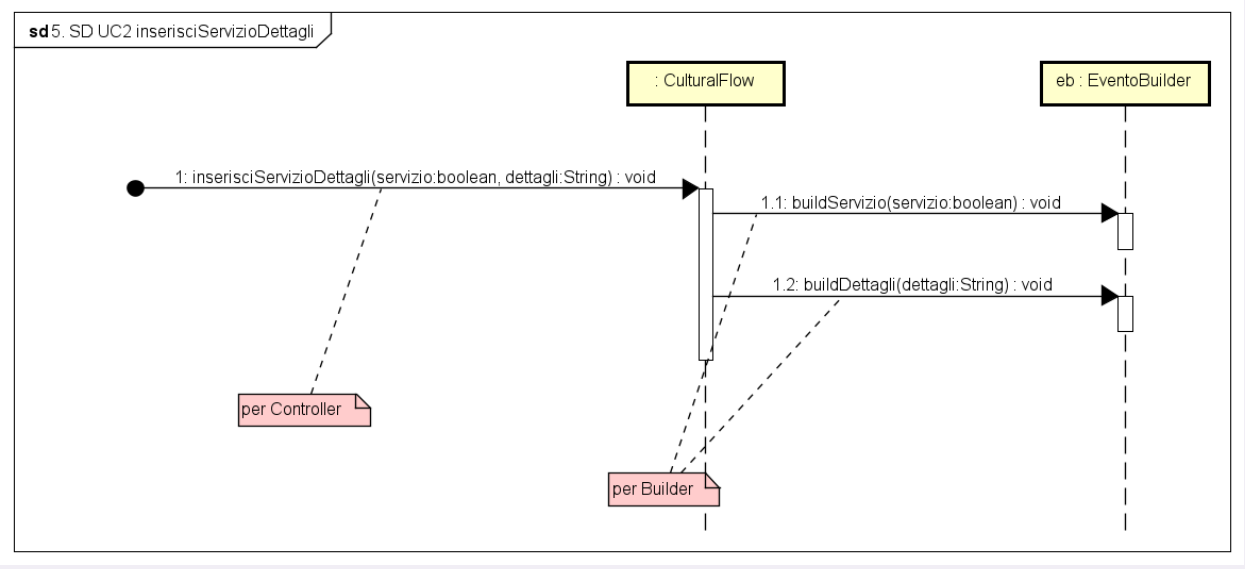
3.3.1. inserisciEvento()



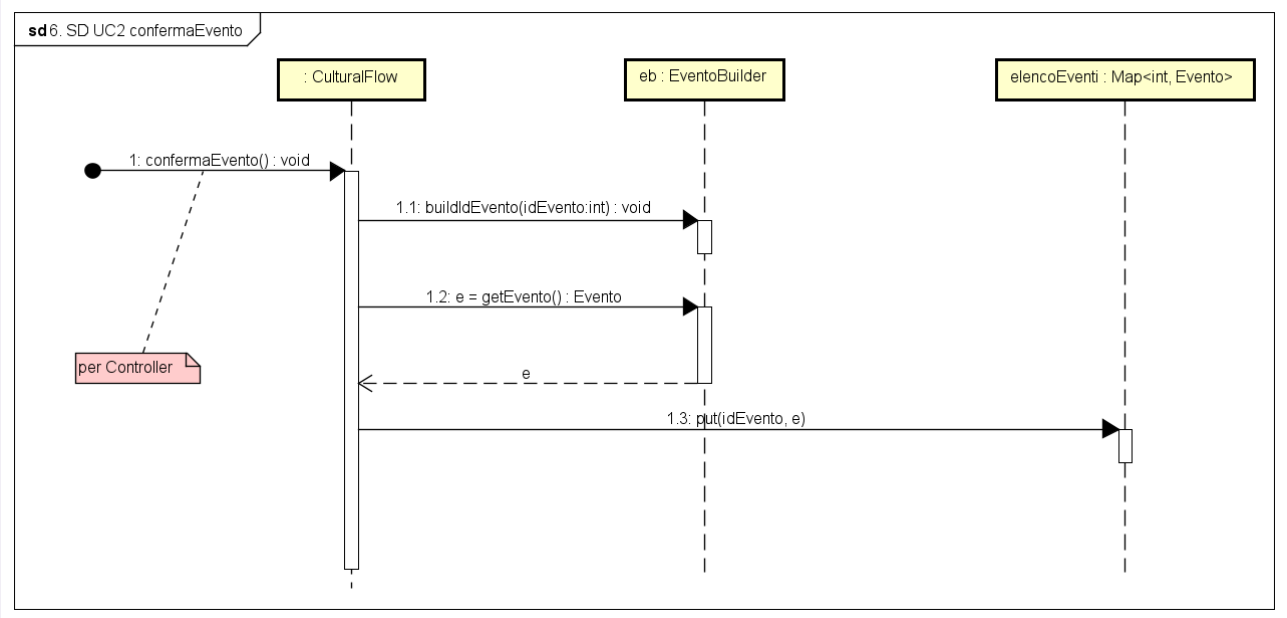
3.3.2. selezionaTipologia()



3.3.3. inserisciServizioDettagli()



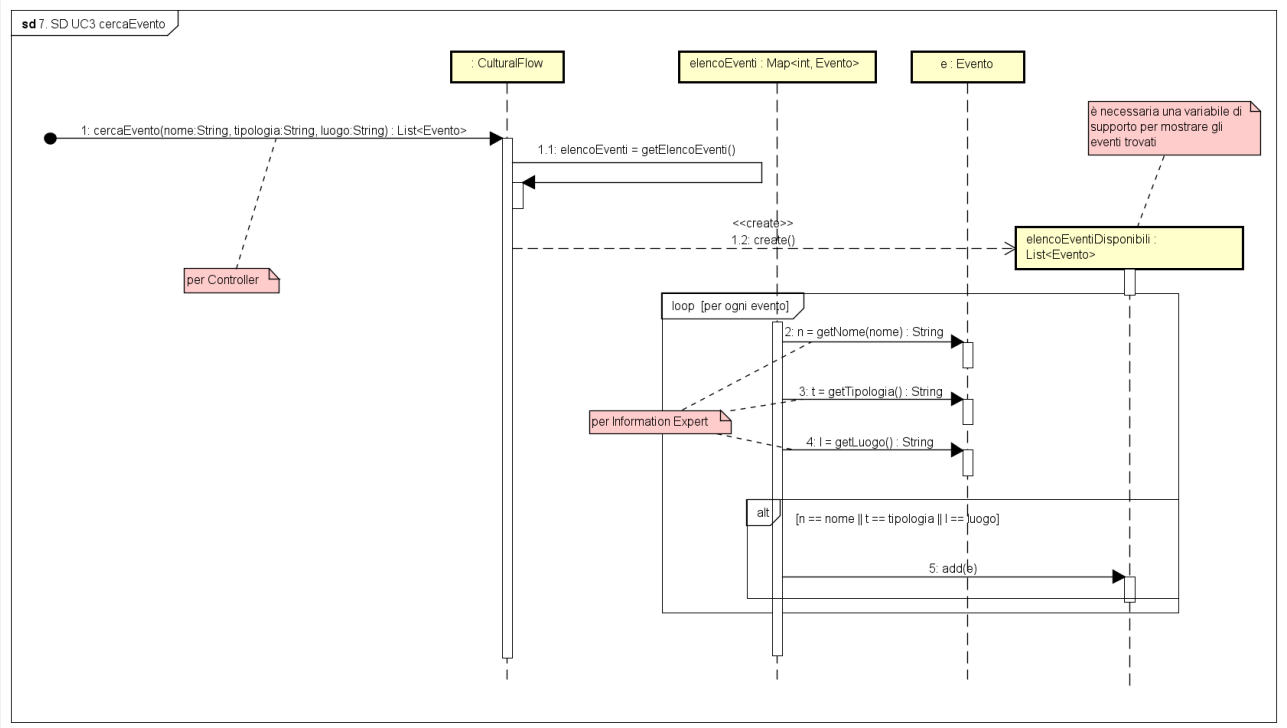
3.3.4. confermaEvento()



3.4. Diagrammi di Sequenza UC3

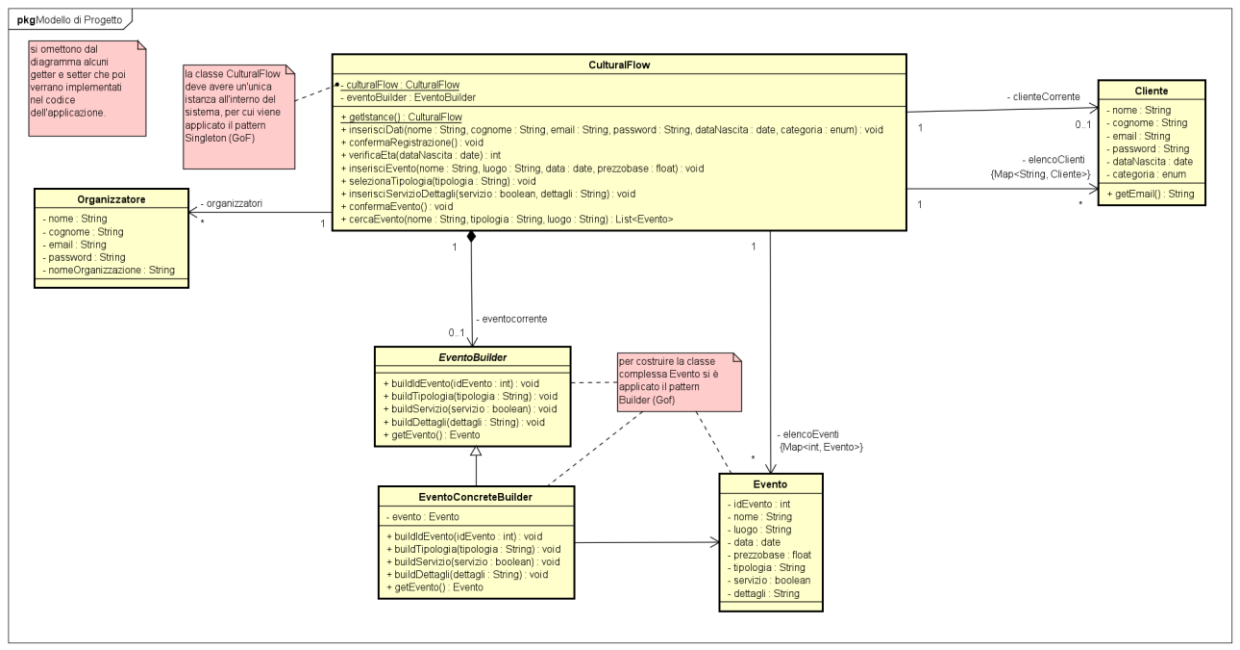
In questo terzo caso d'uso, il sistema CulturalFlow ha la responsabilità di creare l'istanza di elencoEventiDisponibili di tipo List<Evento>. Questo ci permette di avere una struttura dati temporanea per memorizzare e poi mostrare al Cliente i risultati filtrati durante l'esecuzione del metodo cercaEvento. Inoltre, la classe Evento qui agisce come esperto delle informazioni per quanto riguardi i propri attributi, secondo il *pattern Information Expert*.

3.4.1. cercaEvento()



3.4. Diagramma delle Classi (DCD)

Il diagramma delle classi mostra la struttura statica del sistema. Le classi sono state arricchite con i metodi derivati dai contratti delle operazioni e dai diagrammi di sequenza. È possibile individuare i *pattern Singleton* e *Controller*, applicati alla classe CulturalFlow, e il *pattern Builder* usato per la gestione della creazione complessa dell'oggetto Evento.



4. Testing

La fase di test punta a mostrare che il software sia conforme alle sue specifiche e soddisfi le aspettative. Permette la ricerca di errori e anomalie nel software, consentendo così di ridurre i costi di manutenzione e migliorare la qualità del prodotto.

4.1. Individuazione dei casi di test

In questa fase si è svolta un'analisi del codice al fine di individuare le classi e i rispettivi metodi da verificare. È stata data maggiore importanza ai metodi più significativi per il funzionamento del sistema relativamente ai casi d'uso implementati.

Di seguito sono elencati i principali casi di test individuati per le varie classi:

CulturalFlowTest

1. testSingletonInstance()

Verifica che la classe CulturalFlow segua il pattern Singleton, cioè che venga garantita una sola istanza della classe.

2. testInserisciDati()
Verifica che l'inserimento dei dati relativi ad un Cliente funzioni correttamente e che l'istanza, correttamente creata, venga impostata come cliente corrente. Se il Cliente è già registrato, verifica che venga sollevata un'eccezione, testando così l'estensione 6a.

3. testVerificaEta()
Verifica che il Cliente inserito sia effettivamente maggiorenne e solleva un'eccezione nel caso in cui sia minorenne, testando l'estensione 5a.

4. testPassword()
Verifica che la password inserita non sia troppo debole e che venga sollevata un'eccezione in caso contrario.

5. testConfermaRegistrazione()
Verifica che la conferma della registrazione di un Cliente funzioni e che il Cliente venga inserito correttamente nell'elenco dei Clienti.

6. testInserisciEvento()
Verifica che l'inizio del processo di creazione di un Evento funzioni, assicurandosi, inoltre, che venga impostato correttamente come evento corrente.

7. testSelezionaTipologia()
Verifica che la tipologia di evento venga inserita correttamente, confermando che il Builder è in grado di aggiornare la tipologia specifica dell'evento.

8. testInserisciServizioDettagli()
Verifica che gli eventuali servizi e i dettagli di un evento vengano inseriti correttamente.

9. testConfermaEvento()
Verifica che la conferma dell'inserimento di un Evento funzioni, assegna un id univoco e che l'Evento venga inserito nell'elenco degli Eventi.

10. testCercaEvento()
Valida la logica dei filtri di ricerca degli eventi, verificando che la ricerca vada a buon fine.